

Extending memory management tools

I. Motivation

When implementing containers that do not rely on standard allocators it is often necessary to manage memory directly. This paper seeks to fill gaps in the standard library’s memory management utilities.

II. Summary

The function template `destroy` calls the destructor for specified elements.
The function template `uninitialized_move` performs move construction of elements over a range of memory, similar to `uninitialized_copy`.
`uninitialized_move_n` is also provided.
The function template `uninitialized_value_construct` performs value-construction of objects over a range of memory.
The function template `uninitialized_default_construct` performs default-construction of objects over a range of memory.

III. Discussion

Proposed names were approved by LEWG in Kona.

Discussion is underway with Eric Niebler regarding how these algorithms should interact with `range-v3`.

`destroy` first appeared in SGI’s Standard Template Library. It is not known by the author why this algorithm was not inherited into the C++ Standard Library in its initial stages.

It is not possible to implement the “no effects” policy for `destroy` or `uninitialized_move*` so they are specifically excluded from that rule. `uninitialized_move` does need to destroy elements in the output range if an exception is thrown, but the source range cannot be restored to its original state.

Some concern is raised about exception handling with respect to `uninitialized_move`. If a move-constructor throws, moved-from objects may be left in a poorly defined state. Given that algorithm `move` has no special support for this case, it is believed that throwing constructors for this algorithm can be treated similarly.
An additional algorithm, `uninitialized_move_if_noexcept`, could be considered as a resolution to this concern. Such an algorithm was found already implemented in `libstdc++`. Given that there is currently no range-based `move_if_noexcept` algorithm, such a solution is not considered here. It is however trivial to implement such an algorithm – simply forwarding to `copy` or `move` as appropriate.

During implementation research, it was found that `libstdc++` implements a variation of `uninitialized_move` by combining `move_iterator` with `uninitialized_copy`. This approach has some benefit for purposes of wording as it may alleviate needing to provide careful wording for the exception handling behavior of these algorithms. Both wordings are provided as alternatives. Given that the `uninitialized_move` algorithms can be reconstructed simply by combining these two features, the author is willing to drop `uninitialized_move` and `uninitialized_move_n` from this proposal if their motivation is found insufficient.

A design decision must be made regarding the order of destruction in `destroy`. Should bidirectional iterators destroy objects in reverse order? We argue no, for the following reasons:

- It would be a very strange API which requires users to provide reverse iterators in order to destroy in forward order
- Reverse-order destruction can be provided by explicitly providing a reverse iterator. Explicitness is the key here.
- The semantics of an algorithm should not change based on the iterator category. This could lead to surprises during code maintenance.
- Leaving the order as implementation-defined will likely damage its usefulness

It is conceivable that always destroying in forward order would be seen as suprising behavior to some. However, this is at least a consistent behavior which can be accounted for once understood.

IV. Survey of Existing Practice

The following section describes existing implementations of these algorithms prevalent among major C++ implementations. Empty cells denote where no function corresponding to this proposal was found.

Implementations denoted with \$ rely on an additional allocator object to do construction/destruction

P0040	Dinkumware	libstdcxx	libcxx	EASTL	folly (Facebook)
<code>uninitialized_move</code>	\$ <code>_Uninitialized_move</code>	\$ <code>_uninitialized_move_a</code>		<code>uninitialized_move</code>	moveToUninitialized
<code>uninitialized_move_n</code>					
<code>uninitialized_value_construct</code>	\$ <code>_Uninit_def_fill_n</code> (n-variant)	<code>_uninitialized_default</code>	\$ see <code>vector::__construct_at_end</code>	uses <code>uninitialized_fill</code>	see Barrier::allocateControlBlock
<code>uninitialized_default_construct</code>					
<code>destroy</code>	\$ <code>_Destroy_range</code>	_Destroy	\$ see <code>vector::__destruct_at_end</code>	<code>destruct</code>	see ~small_vector

`libstdc++` implements `uninitialized` moves by combining `move_iterator` with `uninitialized_copy`. Most implementations use a separate algorithm instead. The disparity between default and value initialization is to be expected given that standard library containers consistently prefer value initialization for

contained elements.

EASTL's use of uninitialized_fill to implement uninitialized_value_construct is suboptimal in that it requires constructing an additional object, moving it, and destroying it. uninitialized_value_construct and uninitialized_default_construct can be used with non-movable types and may avoid unnecessary operations.

V. Proposed Text

Make the following changes in [specialized.algorithm]:

Modify: In the algorithms uninitialized_copy and uninitialized_move, the template parameter InputIterator is required...

Modify: In the following algorithms other than destroy, uninitialized_move, and uninitialized_move_n, if an exception is thrown there are no effects.

Add: In the algorithms uninitialized_move and uninitialized_move_n, if an exception is thrown then this function has no effects on the range beginning at result.

Add:

```
template<class ForwardIterator>
void destroy(ForwardIterator begin, ForwardIterator end);

Effects:
typedef typename iterator_traits<ForwardIterator>::value_type __t;
while (begin != end)
{
    begin->~__t();
    ++begin;
}

template <class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_move(InputIterator first, InputIterator last, ForwardIterator result);

Effects:
for (; first != last; ++result, ++first)
    ::new (static_cast<void*>(addressof(*result)))
        typename iterator_traits<ForwardIterator>::value_type(std::move(*first));
return result;

template <class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_move_n(InputIterator first, size_t count, ForwardIterator result);

Effects:
for ( ; count>0; ++result, ++first, --count)
    ::new (static_cast<void*>(addressof(*result)))
        typename iterator_traits<ForwardIterator>::value_type(std::move(*first));
return result;

template<class ForwardIterator>
ForwardIterator uninitialized_value_construct(ForwardIterator first, ForwardIterator last);

Effects:
for (; first != last; ++first)
    ::new (static_cast<void*>(addressof(*first)))
        typename iterator_traits<ForwardIterator>::value_type();
return first;

template<class ForwardIterator>
ForwardIterator uninitialized_default_construct(ForwardIterator first, ForwardIterator last);

Effects:
for (; first != last; ++first)
    ::new (static_cast<void*>(addressof(*first)))
        typename iterator_traits<ForwardIterator>::value_type;
return first;
```

Alternative wording

An alternative definition and wording for uninitialized_move is provided as a possible wording simplification. By using move_iterator, we no longer have to tiptoe around the wording for exception handling.

Omit the Add: section above which refers to exceptions in uninitialized_move and uninitialized_move_n

```
template <class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_move(InputIterator first, InputIterator last, ForwardIterator result);

Effects:
return uninitialized_copy( make_move_iterator(first), make_move_iterator(last), result);

template <class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_move_n(InputIterator first, size_t count, ForwardIterator result);

Effects:
return uninitialized_copy_n( make_move_iterator(first), count, result);
```